

Test Strategy

STRATEGY DOCUMENT

Login & New Test Case Features — QA Testing Strategy

DOCUMENT ID TS-001	RELATED PLAN TP-001	VERSION 1.0	STATUS Draft
AUTHOR QA Team	FEATURES Login, New Test Case		

2

FEATURES IN SCOPE

4

TEST TYPES

3

TEST LEVELS

32

TEST CASES (TP-001)

60%

AUTOMATION TARGET



1. Purpose

This Test Strategy defines the overall approach, methodology, and principles governing quality assurance activities for the **Login** and **New Test Case** features. It establishes the testing philosophy and provides direction for all testing decisions made during this release cycle.

This document operates at a higher level than Test Plan TP-001, which specifies the individual test cases. The strategy answers *how* and *why* testing is conducted; the plan answers *what* will be tested.

This strategy should be consulted before any testing decisions are made. Updates to scope, approach, or tooling must be reflected here and communicated to all stakeholders.



2. Quality Objectives

- **Correctness** — All functional requirements for Login and New Test Case are verified to work as specified.
- **Reliability** — Both features behave consistently across repeated interactions and across supported browsers.
- **Security** — The Login page enforces HTTPS and masks all password input; no credentials are exposed in the DOM or network layer.
- **Data Integrity** — Frontend validation prevents malformed or incomplete data from reaching the backend on both features.
- **Usability** — Labels, placeholders, toggles, and error messages meet the stated usability requirements and are unambiguous to the end user.
- **Regression Safety** — High-priority test cases are automated and included in a regression suite to protect against future regressions.



3. Test Levels

System / E2E (QA ownership)

Integration (Shared)

Unit (Dev ownership)

- **System / E2E** — Full user journey testing from UI to backend response. Owned by QA. Covers all 32 test cases in TP-001.
- **Integration** — API contract and frontend-backend interaction testing. Shared between Dev and QA. Covers authentication endpoints and test case submission API.
- **Unit** — Component-level logic validation (e.g. validation functions, toggle state). Owned by development team.

QA is responsible for System/E2E testing. Integration and Unit levels are development responsibilities but QA reviews test coverage reports for those levels before sign-off.



4. Test Types & Coverage Matrix

TEST TYPE	LOGIN	NEW TEST CASE	APPROACH	EXECUTION
Functional	IN SCOPE	IN SCOPE	Black-box; happy path and alternate flows	AUTOMATED
Negative / Validation	IN SCOPE	IN SCOPE	Boundary values, empty fields, invalid formats	AUTOMATED
Security	IN SCOPE	OUT OF SCOPE	HTTPS enforcement, password masking, DOM inspection	MANUAL
Usability	IN SCOPE	IN SCOPE	Label and placeholder review, keyboard behaviour, toggle state	MANUAL
Regression	IN SCOPE	IN SCOPE	Automated High-priority subset; run on every build	AUTOMATED
Performance / Load	OUT OF SCOPE	OUT OF SCOPE	Not in this cycle	—
Accessibility (WCAG)	OUT OF SCOPE	OUT OF SCOPE	Not in this cycle; recommended for next cycle	—
Cross-browser	PARTIAL	PARTIAL	Validation test cases run on Safari, Chrome, Firefox	MANUAL



5. Test Design Techniques

EQUIVALENCE PARTITIONING

- Valid email vs. invalid format classes
- Populated vs. empty required fields
- Remember Me on vs. off session states
- Single step vs. multiple steps

BOUNDARY VALUE ANALYSIS

- Minimum input: one character in fields
- Empty string as boundary for required fields
- Exactly one test step (minimum required)
- Password length extremes

DECISION TABLE TESTING

- Login: valid email + valid pass → redirect
- Login: valid email + wrong pass → error
- Login: empty fields → no backend call
- NTC: all required filled → save success

- NTC: any required empty → error shown

ERROR GUESSING

- Pressing Enter to accidentally submit incomplete forms
- Whitespace-only values in required fields
- Email with valid format but non-existent domain
- Rapid toggle switching on Remember Me / Automated

STATE TRANSITION TESTING

- Login: unauthenticated → authenticated → session expired
- Remember Me: off → on → persisted session
- NTC form: empty → partially filled → fully filled → submitted
- NTC: submitted → visible in list



6. Automation Strategy

SCOPE & TARGET

AUTOMATE (PRIORITY 1)

- All High priority functional test cases
- All High priority validation test cases
- Navigation / redirect flows
- Default toggle states
- Network call verification (no backend call on empty fields)

KEEP MANUAL

- Security visual inspections (HTTPS lock, DOM inspection)
- Remember Me session persistence (requires time passage)
- Label and placeholder visual review
- Toggle visibility and usability
- Cross-browser smoke runs

AUTOMATION TARGET

Target **60% automation coverage** of all 32 test cases in TP-001 for initial release, rising to **80%+** after the first regression cycle. All High priority cases must be automated before the feature is marked regression-ready.

TOOLING

UI AUTOMATION

- Framework: team's existing E2E stack (e.g. Playwright / XCUITest)
- Page Object Model pattern
- Run in CI on every PR merge

NETWORK VERIFICATION

- Intercept / spy on XHR/fetch for frontend validation tests
- Assert no backend call fires when fields are empty

DEFECT MANAGEMENT

- Radar (already in use)
- Auto-attach failure screenshots
- Link failing test ID to radar on creation

Automation scripts must be reviewed and merged to the test repository before the feature is released. No automation debt should be carried forward past the release milestone.



7. Test Data Strategy

LOGIN — REQUIRED DATA

- Valid user account (email + password)
- Invalid credentials (wrong password)
- Non-existent email address
- Admin user account

NEW TEST CASE — REQUIRED DATA

- Valid title strings (short and long)
- Valid expected result strings
- Single test step
- Multiple test steps (3+) in order

- Account with no password set (edge case)
- Whitespace-only inputs (negative)

DATA MANAGEMENT RULES

- **No production data** — all test accounts and test cases must use synthetic, purpose-created data only.
- **Isolation** — test data created during automation runs must not conflict with manually created data; use unique prefixes or dedicated test environments.
- **Cleanup** — automated tests must clean up created test cases after execution to avoid list pollution.
- **Credentials storage** — test credentials must be stored in a secrets manager, never hard-coded in scripts or committed to source control.

8. Test Environment Strategy

ENVIRONMENT REQUIREMENTS

- HTTPS-enabled (mandatory)
- Isolated from production
- Stable build deployed before test cycle
- Admin route accessible
- Forgot Password route accessible

BROWSER COVERAGE

- Safari (primary — macOS)
- Chrome (primary — automation)
- Firefox (validation sanity)
- Mobile Safari (smoke only)

ENVIRONMENT HEALTH CHECKS

- Verify HTTPS certificate valid
- Verify /login route loads
- Verify test credentials are active
- Verify dashboard page accessible post-login

If any health check fails, testing must not commence. Raise a blocker immediately and notify the DevOps team.

9. Entry & Exit Criteria

ENTRY CRITERIA

- **E** Feature implementation is code-complete and deployed to test environment
- **E** HTTPS is confirmed active on the test environment
- **E** Test credentials (valid, invalid, admin) are provisioned
- **E** Test Plan TP-001 reviewed and approved
- **E** This Test Strategy TS-001 reviewed and approved

EXIT CRITERIA

- **X** 100% of High priority test cases executed and passed
- **X** All 3 security test cases (LGN-S-01 to LGN-S-03) passed
- **X** No open Critical or High severity defects in Radar
- **X** 60% automation target met; scripts merged to test repo
- **X** Test results report produced and signed off by QA lead

10. Defect Management

SEVERITY CLASSIFICATION

SEVERITY	DEFINITION	EXAMPLE	RESPONSE
CRITICAL	Security breach or system unusable	Password visible in plain text; HTTPS not enforced	Immediate escalation; block release
HIGH	Core feature broken, no workaround	Login button does not redirect; test case not saved	Fix required before release
MEDIUM	Feature impaired, workaround exists	Error message unclear; placeholder text wrong	Fix targeted for this release; accepted risk if deferred
LOW	Minor cosmetic or edge case	Toggle label spacing; minor UI misalignment	Log and schedule for next cycle

DEFECT LIFECYCLE

- **Tool:** Radar — all defects filed as Radars with linked test case IDs.
- **Required fields:** Title, Steps to Reproduce, Expected Result, Actual Result, Build/Version, Environment, Screenshot or log attachment.
- **Triage:** New radars triaged within 1 business day by QA lead and assigned priority/severity.
- **Re-test:** Fixed defects re-tested by the original tester before closure. A separate re-test note is added to the radar.
- **Security defects:** Any LGN-S-* failure must be escalated immediately regardless of severity label.



11. Metrics & Reporting

TEST EXECUTION RATE Percentage of planned test cases executed by end of cycle. Executed / Total × 100	PASS RATE Percentage of executed test cases that passed on first run. Passed / Executed × 100	DEFECT DENSITY Number of defects found per feature. Tracked by feature and severity. Defects / Feature
AUTOMATION COVERAGE Percentage of test cases covered by automated scripts. Automated TCs / Total TCs × 100	DEFECT RESOLUTION RATE Percentage of filed defects resolved and closed within the cycle. Closed / Filed × 100	RE-OPEN RATE Percentage of closed defects re-opened after re-test. High rate indicates insufficient fix quality. Re-opened / Closed × 100

Metrics are reported at the end of the test cycle in the Test Summary Report. Critical and High defect counts are reported daily to the project lead during active testing.



12. Roles & Responsibilities

ROLE	RESPONSIBILITIES
QA Lead	Owens this strategy and TP-001. Reviews and approves all test cases. Triage defects. Signs off test cycle completion. Reports metrics to stakeholders.
QA Engineer (Manual)	Executes all manual test cases. Files defects in Radar. Performs re-tests on fixed defects. Conducts cross-browser validation runs.

ROLE	RESPONSIBILITIES
QA Engineer (Automation)	Implements automated test cases for all High priority items. Maintains Page Object Models. Integrates automation into CI pipeline. Reviews automation coverage reports.
Development Team	Owens unit and integration test coverage. Resolves filed defects. Supports QA with environment setup and credential provisioning. Reviews defect reports.
DevOps / Infra	Provisions and maintains HTTPS test environment. Confirms environment health before test cycle starts. Supports CI pipeline integration.
Product Owner	Reviews and approves this strategy and TP-001. Accepts risk on deferred Medium/Low defects. Signs off feature readiness for release.

13. Strategic Risks

TECHNICAL RISKS

- HTTPS not configured on test environment
- Frontend validation inconsistent across browsers
- Session/cookie behaviour differs from spec
- Test Case List page dependency not ready
- Network interception not supported in chosen automation framework

PROCESS RISKS

- Test credentials not provisioned before cycle start
- Automation debt carried into release
- Remember Me session window undefined by engineering
- Insufficient time for cross-browser validation runs
- Security defects deprioritised under schedule pressure

Security risks (HTTPS, password exposure) are non-negotiable. No release should proceed with open Critical defects in the LGN-S-* test cases under any circumstance.

14. Related Documents & History

RELATED DOCUMENTS

- TP-001 — Test Plan (Login & New Test Case)
- Feature requirements: Login Feature spec
- Feature requirements: New Test Case spec
- Radar defect tracker

REVISION HISTORY

- v1.0 — Initial draft